TECHNICAL REFERENCE MANUAL

API Documentation & Integration Guide

PDF Vault & Digital Asset Storage System RESTful Interface

Document ID	API-PDFV-2026-V1	**API Version**	1.0.0 (OAS 3.1.0)
Release Date	October 20, 2026	**Base URL**	http://localhost:8000/api/v1
Auth Protocol	OAuth2 Password Bearer (JWT HS256)	**Framework**	FastAPI (Python 3.12+ ASGI)
Format Compatibility	JSON, Multipart/Form-Data, Binary Stream	**Target Audience**	Frontend Engineers, Integrators, Security Auditors

TOTAL ENDPOINTS	AUTH PROTOCOL	MAX FILE PAYLOAD	INTERACTIVE DOCS
16 Routes	**OAuth2 / JWT**	**25 MB**	**/docs & /redoc**
CRUD & streaming routes	Scrypt key derivation	Streaming 1MB buffer	Swagger UI compliant

1. Architecture Overview & Global Conventions

The PDF Vault API is a high-throughput, asynchronous RESTful service built with FastAPI. It provides programmatic access to PDF document archiving, folder organization, image asset catalogs, and user identity boundaries. All operations enforce strict multi-tenant isolation, ensuring users can only interact with their own assets.

- **Protocol & Encoding:** All API interactions use HTTP/1.1 and HTTP/2 over TLS (HTTPS in production). Request and response bodies default to UTF-8 encoded JSON, while file ingestion and file retrieval utilize streaming multipart/form-data and application/octet-stream binaries.
- **API Versioning:** The current active version is exposed under the `/api/v1` namespace. Deprecated endpoints will maintain backwards compatibility for at least two minor version cycles.
- **Interactive Documentation:** Auto-generated OpenAPI 3.1.0 interactive documentation is accessible at `/docs` (Swagger UI) and `/redoc` (ReDoc) for rapid testing and client generation.

2. Authentication & Security Architecture

All protected endpoints require an HTTP Authorization header containing a valid Bearer token issued upon successful login or registration:

```
Authorization: Bearer <eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...>
```

Tokens are cryptographically signed using HMAC-SHA256 against a server secret and expire after 60 minutes (configurable via `AUTH_TOKEN_MINUTES`). The token subject (`sub`) encapsulates the user's UUID.

HTTP Status	Error Condition	Response Body Structure
401 Unauthorized	Missing, expired, or tampered Bearer token	{"detail": "Invalid or expired authentication token."}
403 Forbidden	Attempting to access unauthorized tenant resources	{"detail": "Forbidden: Access denied to foreign resource."}
413 Payload Too Large	Uploaded file exceeds 25 MB ceiling	{"detail": "Maximum upload size is 25 MB."}
415 Unsupported Media Type	File signature or MIME validation fails	{"detail": "File does not have a valid PDF header."}
422 Unprocessable Entity	Pydantic payload schema validation failure	{"detail": [{"loc": ["body", "email"], "msg": "field required"}]}

3. System & Health Probes

Lightweight operational monitoring routes for load balancers and orchestrators:

3.1 Service Root Probe

- **Endpoint:** `GET /` (Public)
- **Summary:** Returns the root system greeting and documentation path.

- **Response Code:** 200 OK

```
// Response Body:
{
  "message": "PDF Vault API is running",
  "docs": "/docs"
}
```

3.2 Health Check Probe

- **Endpoint:** `GET /api/v1/health` (Public)
- **Summary:** Liveness and readiness probe for Kubernetes / Cloud Run container health.
- **Response Code:** 200 OK

```
// Response Body:
{
  "status": "ok",
  "service": "PDF Vault API"
}
```

4. Identity & Authentication API (`/api/v1/auth`)

4.1 Register New Account

- **Endpoint:** `POST /api/v1/auth/register` (Public)
- **Description:** Registers a new user account with unique username and email. Hashes password using Scrypt.
- **Request Headers:** `Content-Type: application/json`

```
// Request Body:
{
  "username": "alex_developer",
  "email": "alex@enterprise.com",
  "password": "SecurePassword123!"
}
```

- **Response (201 Created):** Returns the newly generated access token and sanitized user profile:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user": {
    "id": "7b82f6e9-b2b9-4a41-b844-3112c321876a",
    "username": "alex_developer",
    "email": "alex@enterprise.com"
```

```
  }
}
```

4.2 User Authentication / Login

- **Endpoint:** `POST /api/v1/auth/login` (Public)
- **Description:** Authenticates credentials conforming to OAuth2 password specification. The username field expects the user's email.
- **Request Headers:** `Content-Type: application/x-www-form-urlencoded`

```
// Request Form Payload:
username=alex%40enterprise.com&password=SecurePassword123!
```

- **Response (200 OK):** Returns signed bearer token:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user": {
    "id": "7b82f6e9-b2b9-4a41-b844-3112c321876a",
    "username": "alex_developer",
    "email": "alex@enterprise.com"
  }
}
```

4.3 Get Current User Profile

- **Endpoint:** `GET /api/v1/auth/me` (Authenticated)
- **Description:** Retrieves active user profile claims based on the provided Bearer token.
- **Response (200 OK):** User object:

```
{
  "id": "7b82f6e9-b2b9-4a41-b844-3112c321876a",
  "username": "alex_developer",
  "email": "alex@enterprise.com"
}
```

5. Document Vault & Folders API (`/api/v1/documents`)

5.1 List Document Folders

- **Endpoint:** `GET /api/v1/documents/folders` (Authenticated)

- **Description:** Returns all folders owned by the authenticated user, sorted alphabetically.

```
// Response (200 OK):
[
  { "id": "2d1f6a8e-4a11-409e-8c9f-3e8a712f551b", "name": "Contracts 2026" },
  { "id": "9c8a14b2-e1a3-4b67-91fa-2849e771ad20", "name": "Tax Invoices" }
]
```

5.2 Create Document Folder

- **Endpoint:** `POST /api/v1/documents/folders` (Authenticated)
- **Request Body:** `{"name": "Legal Briefs"}` (1–120 characters)
- **Response (201 Created):** `{"id": "c4b12a88-...", "name": "Legal Briefs"}`
- **Error Responses:** 409 Conflict if folder name already exists for user; 422 Unprocessable Entity if name is empty.

5.3 Delete Document Folder (Safe Unlink)

- **Endpoint:** `DELETE /api/v1/documents/folders/{folder_id}` (Authenticated)
- **Description:** Deletes the folder record and safely resets all child documents' `folder_id` to NULL. Physical documents are NOT deleted.
- **Response:** 204 No Content

5.4 List & Search Documents

- **Endpoint:** `GET /api/v1/documents` (Authenticated)
- **Query Parameters:** `search` (optional string, case-insensitive ILIKE match), `folder_id` (optional UUID)

```
// Response (200 OK):
[
  {
    "id": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "filename": "Q3_Financial_Audit.pdf",
    "size_bytes": 1048576,
    "created_at": "2026-10-20T14:32:00Z",
    "folder_id": "2d1f6a8e-4a11-409e-8c9f-3e8a712f551b"
  }
]
```

5.5 Upload PDF Document (Binary Magic-Byte Inspection)

- **Endpoint:** `POST /api/v1/documents` (Authenticated)
- **Content-Type:** `multipart/form-data`

- **Form Fields:** `file` (UploadFile binary), `folder_id` (optional string UUID)
- **Validation Pipeline:** 1. Filename ends with `.pdf`. 2. Stream chunked in 1MB buffers. 3. Total bytes <= 25MB. 4. Binary header matches `b'%PDF-'` exactly.
- **Response (201 Created):** Returns metadata of stored PDF:

```
{
  "id": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
  "filename": "Q3_Financial_Audit.pdf",
  "size_bytes": 1048576,
  "created_at": "2026-10-20T14:32:00Z",
  "folder_id": "2d1f6a8e-4a11-409e-8c9f-3e8a712f551b"
}
```

5.6 Download PDF Document

- **Endpoint:** `GET /api/v1/documents/{document_id}/download` (Authenticated)
- **Response Code:** 200 OK
- **Response Headers:** `Content-Type: application/pdf`, `Content-Disposition: attachment; filename="Q3_Financial_Audit.pdf"`

5.7 Delete PDF Document

- **Endpoint:** `DELETE /api/v1/documents/{document_id}` (Authenticated)
- **Description:** Atomically deletes the database row and permanently unlinks the stored file `<uuid>.pdf` from the server filesystem.
- **Response:** 204 No Content

6. Image Asset Studio API (`/api/v1/images`)

6.1 Image Categories Management

- **List Categories:** `GET /api/v1/images/categories` -> 200 OK array of `{id, name}`
- **Create Category:** `POST /api/v1/images/categories` -> 201 Created `{id, name}`
- **Delete Category (Cascading):** `DELETE /api/v1/images/categories/{id}` -> 204 No Content (deletes all child images and disk files)

6.2 Image Upload & Retrieval

- **List Images in Category:** `GET /api/v1/images?category_id={id}&search={term}` -> 200 OK array of image metadata

- **Upload Image Asset:** `POST /api/v1/images` (Multipart form: `file`, `category_id`). Enforces approved MIME (`image/jpeg`, `image/png`, `image/gif`, `image/webp`, `image/bmp`) and <= 25MB.
- **Stream Image File:** `GET /api/v1/images/{image_id}/file` -> 200 OK with binary stream and original image content-type for preview rendering.
- **Delete Image Asset:** `DELETE /api/v1/images/{image_id}` -> 204 No Content

7. Client Integration Code Examples

Direct implementation patterns for client applications:

7.1 Python Integration (`httpx` / `requests`)

```
import httpx

BASE_URL = "http://localhost:8000/api/v1"

# 1. Authenticate and retrieve token
client = httpx.Client(base_url=BASE_URL)
login_res = client.post("/auth/login", data={"username": "alex@enterprise.com", "password":
"SecurePassword123!"})
token = login_res.json()["access_token"]
headers = {"Authorization": f"Bearer {token}"}

# 2. Upload a verified PDF file
with open("annual_report.pdf", "rb") as f:
    files = {"file": ("annual_report.pdf", f, "application/pdf")}
    data = {"folder_id": "2d1f6a8e-4a11-409e-8c9f-3e8a712f551b"}
    upload_res = client.post("/documents", headers=headers, files=files, data=data)
    print("Uploaded Document ID:", upload_res.json()["id"])
```

7.2 JavaScript / TypeScript Integration (`fetch`)

```
async function uploadPdf(token, fileObject, folderId) {
  const formData = new FormData();
  formData.append('file', fileObject);
  if (folderId) formData.append('folder_id', folderId);

  const response = await fetch('http://localhost:8000/api/v1/documents', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`
    },
    body: formData
  });

  if (!response.ok) {
    const errorData = await response.json();
```

```
    throw new Error(errorData.detail || 'Upload failed');
  }

  return await response.json();
}
```